Software Architecture and Design Specification (SAD)

Project: Traffic Management System (Simulation)

Version: 1.0

Authors: Aarya, Vijeth, Vaibhav, Vainik

Date: 12-10-2026

Status: Final – for team review

Approvals

Role	Name	Signature/Date
Course Coordinator		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Traffic Management System (TMS) Simulation, a C++17 discrete-time traffic simulator.

1.2 Scope

Covers scenario loading, vehicle generation and queues, adaptive signal control, road network and routing, movement, congestion and incident handling, metrics and reporting through a console interface.

1.3 Audience

Developers, testers, reviewers, instructors and maintainers.

1.4 Definitions

TMS, SRS, FR, NFR, RTM, Tick, Lane Queue, Signal Phase, RAII, ADR, STRIDE.

2. Document Overview

2.1 How to use this document

Section 3 describes architecture (components, pattern, traceability, security). Section 4 describes design (sequence diagrams, API, error handling).

2.2 Related Documents

TMS SRS v1.0, TMS Software Test Plan v1.0, project plan in the repository.

3. Architecture

3.1 Goals & Constraints

- Goals: modular, testable, deterministic, safe (no conflicting greens), reproducible.
- Constraints: C++17 and CMake on Linux; in-process modules only; no real hardware; no mutable globals except a controlled simulation context (NFR-01, NFR-05).

3.2 Stakeholders & Concerns

Stakeholder	Concerns
Scenario Designer / Operator	Usability, clear errors, repeatable runs
Traffic Analyst	Correct metrics and reports
Developers / Testers	Modularity, testability, deterministic behaviour
Evaluators	Traceability to SRS, quality, security

3.3 Component (UML) Diagram

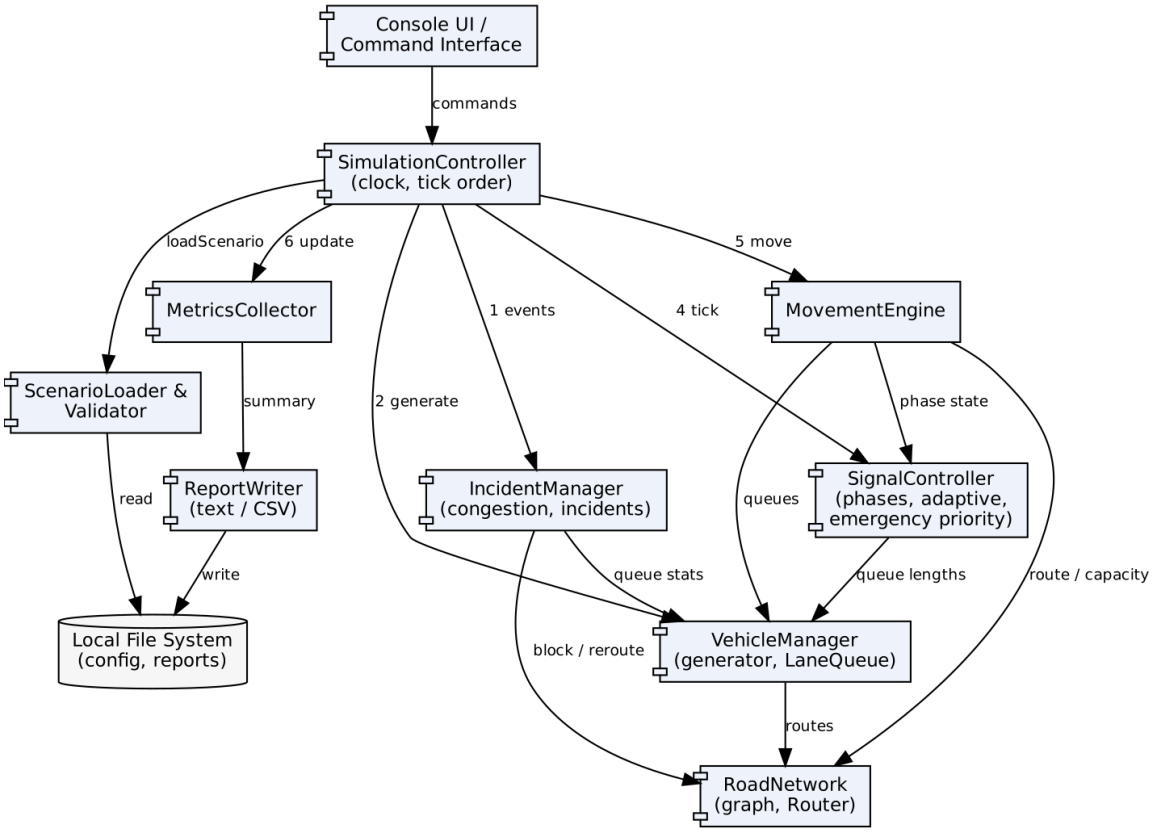


Figure 1: Component diagram. Numbers on SimulationController edges show the fixed tick-processing order.

3.4 Component Descriptions

Component	Responsibility	Key operations
Console UI	Menu/command interface, input prompts, status display	showMenu(), parseCommand()
SimulationController	Owns clock, tick order, run/stop/reset; the controlled simulation context and seeded RNG	loadScenario(), step(), run(), reset()
ScenarioLoader & Validator	Reads and validates scenario files	load(path), validate()
VehicleManager (Vehicle, LaneQueue)	Generates vehicles, assigns IDs/routes, enforces entry-lane capacity	generateVehicle(), insertAtEntry(), validateVehicle()
SignalController	Phases, timers, adaptive green, emergency priority; never enables conflicts	tick(), requestPriority(), isGreen()
RoadNetwork (Road, Router)	Graph of nodes/edges, capacities, availability, shortest-path routing	addRoad(), neighbors(), findRoute(), isRoadAvailable()
MovementEngine	Moves vehicles when signal, route and capacity allow; completes trips	advanceVehicles(), crossIntersection(), enterRoad(), completeTrip()
IncidentManager	Congestion detection, incident scheduling/clearing, triggers rerouting	scheduleIncident(), applyIncident(), clearIncident(), congestionStatus()
MetricsCollector	Throughput, waiting time, queue statistics, trip counts	recordWait(), recordDeparture(), buildSummary()
ReportWriter	Text and CSV export	writeText(), writeCsv()

3.5 Chosen Architecture Pattern and Rationale

Pattern: Modular, layered architecture with a central orchestrator (SimulationController) and a time-stepped (tick-based) simulation loop. Layers: Presentation (Console UI) → Orchestration (SimulationController) → Domain modules (Vehicle, Signal, Network, Movement, Incident) → Services (Metrics, Report, file I/O).

Rationale: separates concerns, lets each team member own one module, allows each module to be unit-tested through its interface (NFR-01, NFR-02), and keeps a single stable tick order that makes runs deterministic (FR-18). Rejected alternatives: event-driven/actor model (more complex, harder to make deterministic); microservices (unnecessary for a standalone simulator).

ADR-1: Use Dijkstra on an adjacency list (small sparse graph). ADR-2: Single seeded std::mt19937 in the simulation context for reproducibility. ADR-3: Fixed tick order – (1) apply incidents, (2) generate vehicles, (3) update congestion, (4) tick signals, (5) move vehicles in deterministic order, (6) update metrics, (7) advance clock.

3.6 Technology Stack & Data Stores

Concern	Choice
Language / build	C++17, CMake
Testing	GoogleTest, CTest, gcov/lcov
Quality	clang-format, clang-tidy/cppcheck, ASan/UBSan
CI/CD	GitHub Actions now (build, test, static analysis on every PR); Jenkins pipeline, Docker image and SonarQube planned per course guidelines
Data structures	std::deque (lane queues), adjacency list (graph), std::priority_queue (Dijkstra), std::unordered_map (registries)
Persistence	Text scenario files in config/; text/CSV reports on local file system (no database)

3.7 Risks & Mitigations

Risk	Mitigation
Inconsistent module integration	Documented fixed tick order; integration tests
Non-determinism	Single seeded RNG; ordered containers for iteration
Safety violation (conflicting greens)	Conflict matrix checked inside SignalController before any phase change; assertions in tests
Invalid input crashes the program	Central validation in ScenarioLoader and Console UI (NFR-04, NFR-07)

3.8 Traceability to Requirements

Requirement(s)	Component(s)
FR-01, NFR-04	ScenarioLoader & Validator, SimulationController
FR-02, FR-03, FR-04	VehicleManager (Vehicle, LaneQueue)
FR-05, FR-06, FR-07, FR-08	SignalController
FR-09, FR-12	RoadNetwork (Router)
FR-10, FR-11	MovementEngine
FR-13, FR-14, FR-15	IncidentManager
FR-16	MetricsCollector
FR-17	ReportWriter
FR-18, NFR-10	SimulationController (seeded RNG)
NFR-01, NFR-02, NFR-08	All modules via interfaces; RTM
NFR-03	Simulation loop, MovementEngine, Router
NFR-05, NFR-09	CMake, CI, static analysis
NFR-06	Console UI, ReportWriter
NFR-07, SEC-01 to SEC-04	Console UI and ScenarioLoader validation, SignalController safety check, RAII throughout

3.9 Security Architecture

The system is a local, standalone, single-user simulator, so the main threats come from malformed inputs and misuse of commands. Threat model (STRIDE):

Threat	Example	Control	Req
Spoofing	Not applicable (no authentication, local use)	N/A – documented assumption	–
Tampering	Edited scenario file with impossible values	Schema and range validation; reject on any error	SEC-01, SEC-02
Repudiation	Untraceable run	Log seed and config in every report	NFR-10
Information disclosure	Path leaks in error messages	Messages state cause only, no internal paths/memory	NFR-06
Denial of service	Huge file or extreme parameters hang the run	Size and value limits; capacity/duration caps	SEC-01, SEC-02
Elevation of privilege / safety bypass	Emergency request forces conflicting greens	Conflict check inside SignalController, cannot be bypassed by callers	SEC-04

Security objectives: SO-1 Integrity, SO-2 Availability, SO-3 Safety. Secure coding: RAII, std::string/std::vector only, no raw owning pointers, compiler warnings and sanitizers in CI (SEC-03).

3.10 Division of Work and Module Ownership

Member	Module ownership	Requirements	Interfaces documented
Aarya	Vehicle, LaneQueue, VehicleManager, input	FR-01 to FR-04	IVehicleManager

	validation		
Vijeth	SignalPhase, SignalController, signal policy	FR-05 to FR-08	ISignalController
Vaibhav	Road, RoadNetwork, Router, MovementEngine	FR-09 to FR-12	IRoadNetwork
Vainik	IncidentManager, MetricsCollector, ReportWriter	FR-13 to FR-17	IMetricsCollector, IReportWriter
All	SimulationController integration, FR-18, NFRs, CI	FR-18, NFR-01 to NFR-10	Tick-order contract

3.11 Interface Stability and ABI

The TMS builds as a single statically linked C++17 executable (plus test binaries), so there is no shared-library ABI to preserve. Stability is managed at source (API) level: public headers under include/<module>/ are the only cross-module contract; changes to them need a PR approved by the module owner and a reviewer. Interfaces are pure abstract classes (virtual destructor, no data members, no templates in signatures that cross modules) so tests can substitute mocks. If a module is later built as a shared library, only the abstract interfaces and plain value types (ids, Route, Vehicle) would form the ABI, and compiler/standard-library versions would have to match across modules.

4. Design

4.1 Design Overview

Each module is a C++ class (or small group) behind an interface in include/<module>/. The SimulationController calls modules in the fixed tick order. Data is exchanged via validated value objects (Vehicle, Road, Phase), not shared globals.

4.2 UML Sequence Diagrams

Figure 2 – Sequence 1 – Normal tick: vehicle generation, signalling and movement (FR-02 to FR-06, FR-10, FR-11, FR-16).

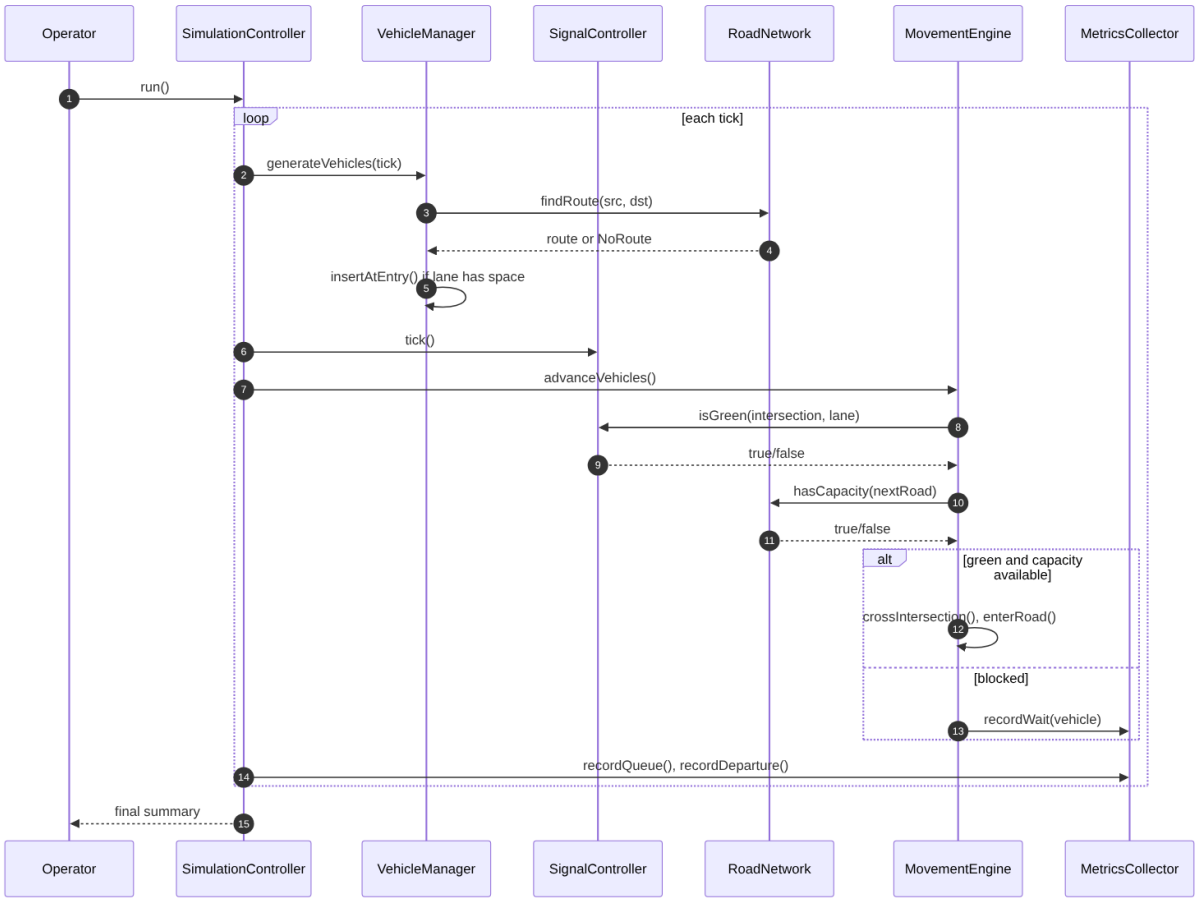
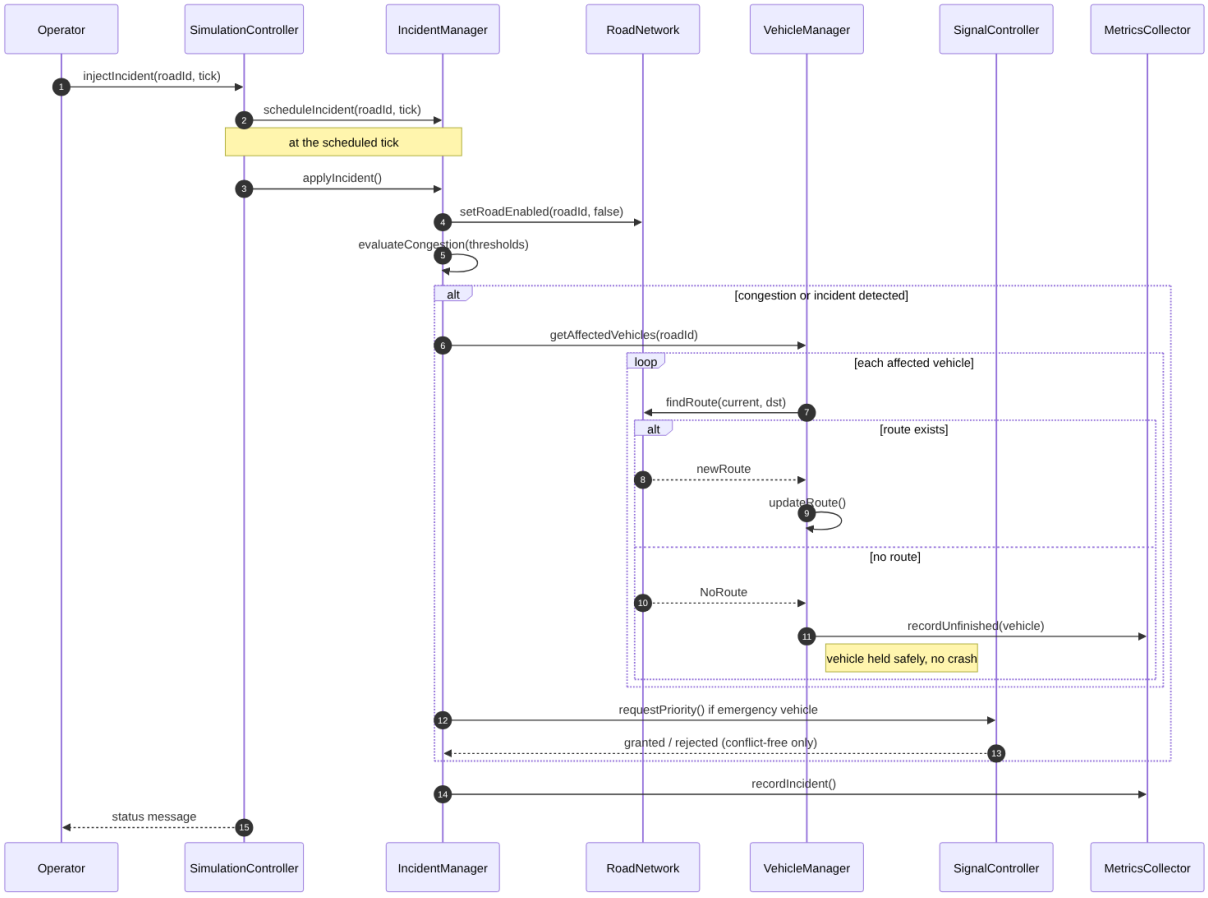


Figure 3 – Sequence 2 – Incident injection, congestion detection and rerouting (FR-12 to FR-15).



4.3 API Design

Because all communication is in-process (see SRS "Communications"), APIs are C++ module interfaces. Two key interfaces:

Interface	Method	Inputs	Returns / Errors
IRoadNetwork	findRoute(src, dst)	NodeId, NodeId	Result<Route> – error NoRoute, UnknownNode
IRoadNetwork	hasCapacity(roadId)	RoadId	bool; error UnknownRoad
IRoadNetwork	setRoadEnabled(roadId, bool)	RoadId, bool	void; error UnknownRoad
ISignalController	tick()	–	void
ISignalController	isGreen(intersectionId, laneId)	ids	bool
ISignalController	requestPriority(intersectionId, approach)	id, approach	Granted / Rejected(ConflictingPhase)
IVehicleManager	generateVehicles(tick)	tick	vector<VehicleId>
IVehicleManager	insertAtEntry(vehicle)	Vehicle	Ok / Rejected(LaneFull)
IMetricsCollector	recordWait / recordDeparture / buildSummary	vehicleId, tick	Summary
IReportWriter	writeCsv(path, summary)	path, Summary	Ok / Error(IoFailure)

Console command interface: load <file>, run [ticks], status, incident <road> <tick>, priority <intersection> <approach>, stats, export <path>, quit.

4.4 Error Handling, Logging & Monitoring

- Errors are returned as typed results (e.g. std::variant / custom Result or enum) at module boundaries; exceptions used only for fatal setup failures and caught in main.
- Standard error format: [ERROR][Module][Code] message. Example: [ERROR][Scenario][E101] Road R3 capacity must be > 0.
- Invalid configuration: reject, leave state unchanged, report cause, exit code non-zero for batch runs (NFR-04).
- No-route and full-lane situations are normal outcomes, not crashes (FR-11, FR-15).
- Logging: levels INFO/WARN/ERROR to console and optional log file; seed and scenario name logged at start; no sensitive or internal path data.
- Monitoring: per-run counters (vehicles generated, completed, unfinished, max queue, rejected inputs) in the final report.

Code	Meaning
E1xx	Scenario/configuration error
E2xx	Vehicle/queue error (lane full, invalid vehicle)
E3xx	Signal error (conflicting phase request)
E4xx	Routing/network error (no route, unknown id)
E5xx	I/O error (file read/write)

4.5 UX Design

Console menu with numbered options, consistent labels, concise error messages and a readable summary table (NFR-06).

4.6 Module Design: Road Network, Routing and Movement

This module (FR-09 to FR-12) is the core of vehicle progression across the network.

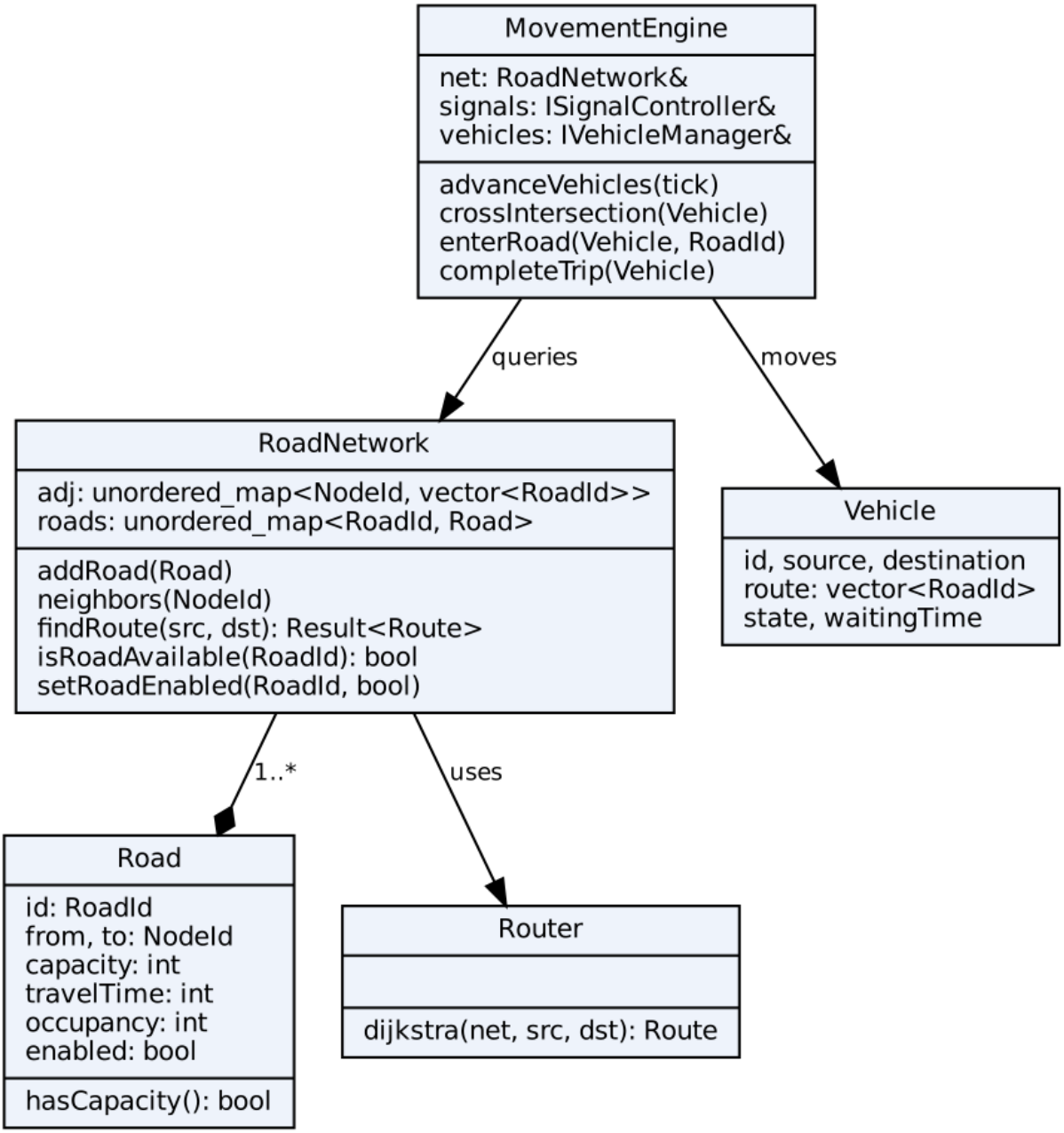


Figure 4: Class diagram of the Road Network and Movement module.

- Graph: adjacency list (unordered_map<NodeId, vector<RoadId>>), roads stored by id. Sparse graph, O(1) neighbour lookup (FR-09).
- Routing: Dijkstra with std::priority_queue; edge cost = travelTime (weighted-path policy). Disabled roads are skipped, so blocked roads are avoided automatically. Complexity O((V+E) log V). Unreachable destination returns NoRoute (FR-12, FR-15).
- Movement rule per tick, evaluated in deterministic order (ascending intersection id, then lane id): a vehicle at the front of its lane moves only if (a) its signal is green, (b) the next road on its route is enabled, and (c) the next road occupancy < capacity. Otherwise it waits and its waiting time increases (FR-10, FR-11).

- Trip completion: when a vehicle reaches its destination node it is removed, its trip time is recorded and the completed counter increments.
- Invariants: occupancy <= capacity for every road and lane at the end of every tick; a vehicle is in exactly one place (lane queue, road or completed).

4.7 Open Issues & Next Steps

- Finalise conflict matrix format for signal phases.
- Agree Result/error type shared by all modules.
- Optional: simple ASCII visualisation of queues.

5. Appendices

5.1 Glossary

See SRS Definitions and Acronyms.

5.2 References

IEEE 42010, IEEE 1016, OWASP Secure Coding Practices, NIST SP 800-160.

5.3 Tools

Graphviz, Mermaid, draw.io, GoogleTest, CMake, GitHub Actions.